Load Testing That Actually Tests Load

COMPSCI 426

- Why last time's load test didn't work
- Fixing the server and the test
- Observing scaling under real pressure
- Breaking the system and finding limits

Today

1. What went wrong last time
2. What we changed
3. Activity: three scenarios
4. Scaling experiment debrief
5. Closing

Learning Goals

By the end of class, you should be able to:

- explain why a load test can fail to reveal architecture differences
- distinguish blocking (CPU-bound) from non-blocking (async) server work
- design a k6 test that saturates a server
- interpret scaling results and identify resource ceilings

The Problem

> We ran three architectures through k6 last time. Single replica. Three replicas. Three replicas with a load balancer. The numbers were nearly identical. Why?

Problem 1: The Test Throttled Itself

```
export const options = { vus: 20, duration: '30s' };
export default function () {
  http.get(targetUrl);
  sleep(1);  // each VU waits 1 full second between requests
}
```

- Each VU: one request (~200ms), then **1 second idle**
- 20 VUs = ~16 req/s max
- The server was never stressed
- **Bottleneck: the test client, not the server**

Problem 2: The Server Work Was Non-Blocking

```
await sleep(delayMs);  // setTimeout -- does NOT block the event loop
```

- `setTimeout` is async: Node.js handles thousands concurrently
- 20 VUs sleeping for 200ms? Trivial for one process
- No queuing, no contention, no reason for replicas to help

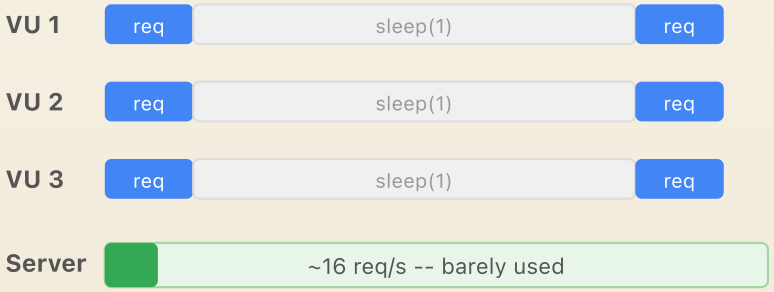
If the server can handle everything thrown at it, extra replicas have nothing to fix.

Last Time vs Today

Why the Old Load Test Did Not Stress the Server

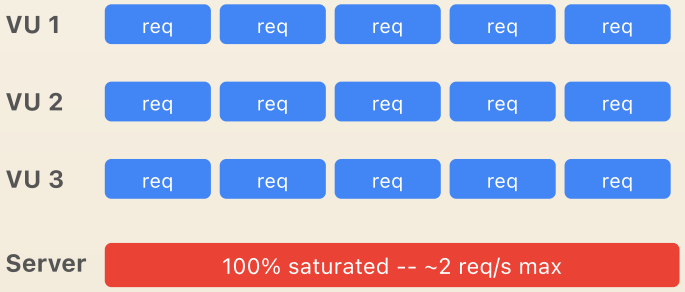
Last Class: Client-Throttled

20 VUs, sleep(1) between requests



Today: Server-Saturated

100 VUs, no sleep between requests



Key Insight

On the left, VUs spend 83% of their time sleeping. The server is idle most of the time. Adding replicas to an idle server changes nothing measurable.

On the right, VUs send requests back-to-back. The server is the bottleneck. Now adding replicas behind a load balancer produces a visible improvement.

Fix 1: CPU-Bound Blocking Work

```
function cpuWork(ms) {
  const end = Date.now() + ms;
  while (Date.now() < end) {}  // blocks the event loop
}
```

- Occupies the event loop for the full 500ms
- One request at a time per process
- Max throughput: ~2 req/s per replica

This simulates real CPU work: image processing, encryption, computation.

Fix 2: No Sleep, More VUs

```
export const options = { vus: 100, duration: '30s' };
export default function () {
  http.get(targetUrl);
  // no sleep -- next request immediately
}
```

- 100 VUs sending requests as fast as the server can respond
- The server is now the bottleneck, not the client
- With 1 replica: ~2 req/s, massive queuing
- With 3 replicas + LB: ~6 req/s, 1/3 the latency

Service Level Objectives (SLOs)

How do you know if your system is "fast enough"?

- **SLI** (Service Level Indicator): a metric you measure – latency, error rate, throughput
- **SLO** (Service Level Objective): a target for that metric – "p95 latency < 500ms"
- **SLA** (Service Level Agreement): a contract with consequences if the SLO is missed

SLOs turn subjective feelings ("it feels slow") into objective measurements.

You can have SLOs without SLAs. Most internal services define SLOs to guide engineering decisions.

SLOs in Practice

An SLO has three parts:

1. **The SLI**: what you measure (e.g., request latency)
2. **The target**: what "good enough" means (e.g., p95 < 500ms)
3. **The window**: the time period (e.g., rolling 24 hours)

In k6, thresholds are SLOs:

```
thresholds: {
  http_req_duration: ['p(95)<2000'],  // p95 latency under 2s
  http_req_failed: ['rate<0.10'],     // error rate under 10%
}
```

When a threshold fails, the system violated that SLO under load. This tells you your **capacity limit** – when to scale.

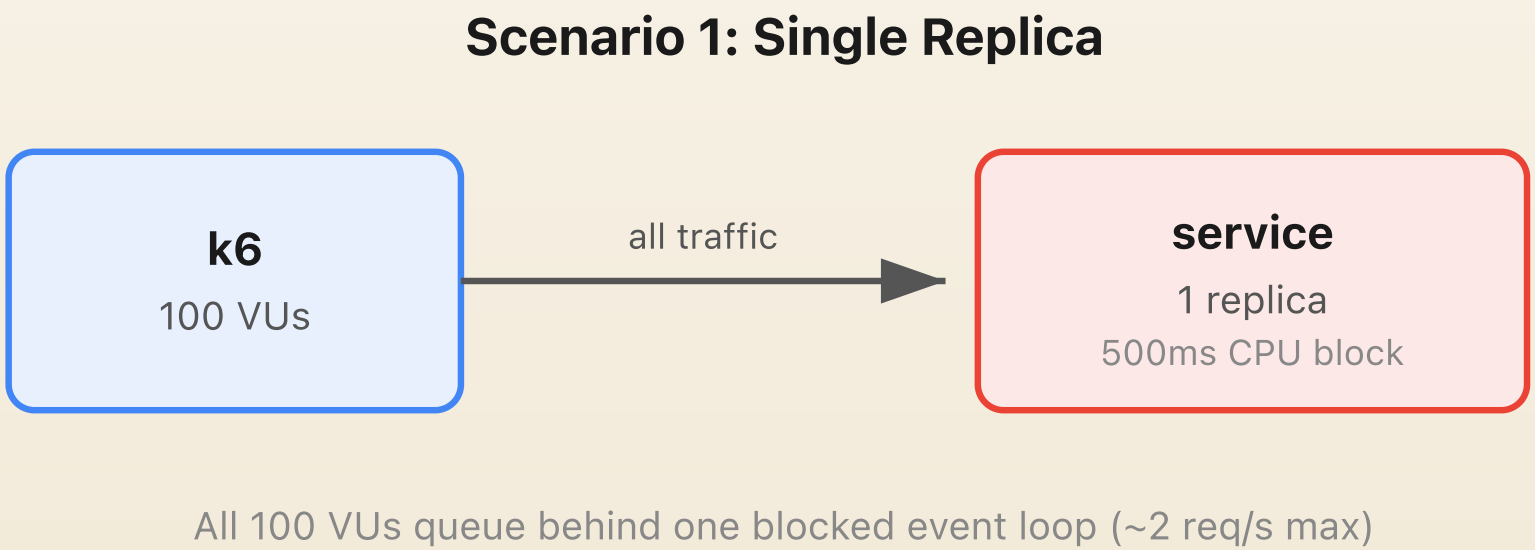
Activity Overview

Three scenarios, same pattern as last time, but now the differences are measurable:

1. **Scenario 1**: 1 replica, CPU-bound – your baseline
2. **Scenario 2**: 3 replicas + Caddy load balancer – expect ~3x throughput
3. **Scenario 3**: Ramping to 800 VUs – find the breaking point

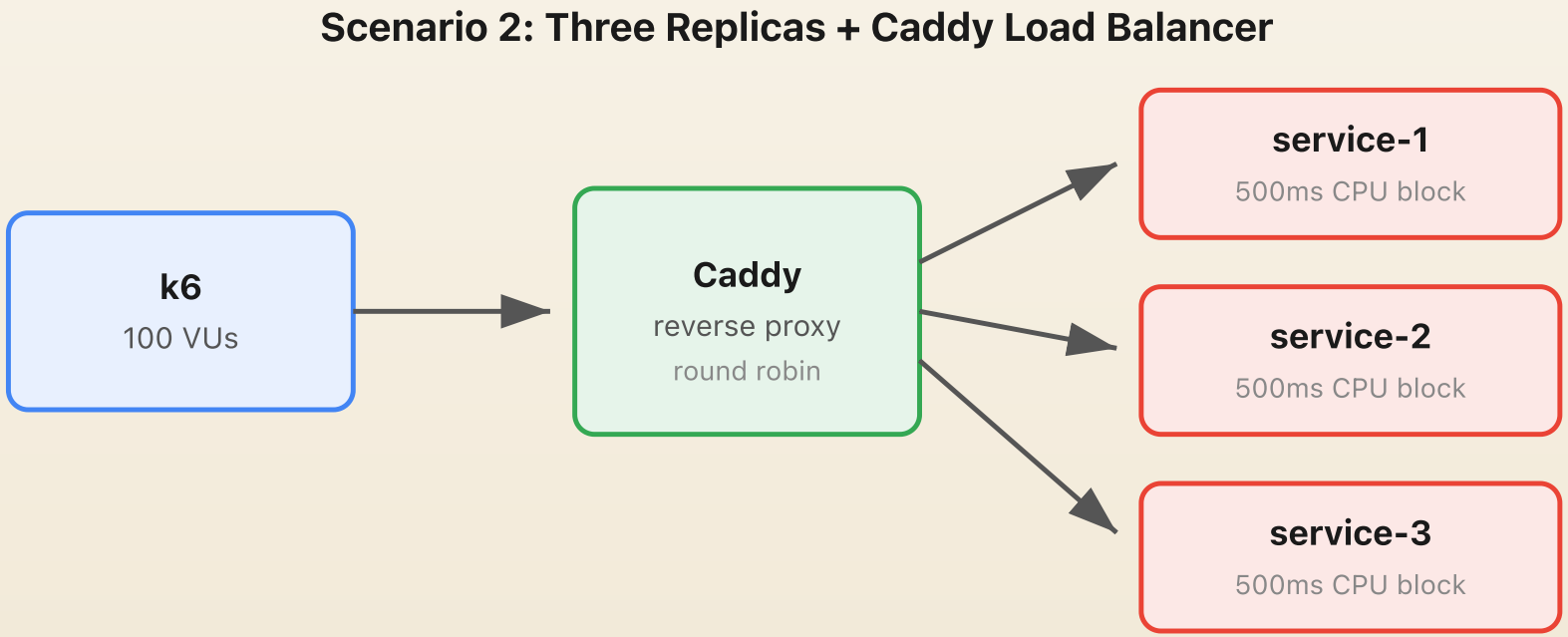
Plus a scaling experiment: what happens as you go from 5 to 30 replicas?

Scenario 1: Single Replica



- 100 VUs → 1 process that blocks for 500ms per request
- Max throughput: ~2 req/s
- Expect very high latency

Scenario 2: Caddy + 3 Replicas



Caddy distributes requests across 3 event loops. ~6 req/s total, ~3x Scenario 1.

- Caddy round-robins across 3 independent event loops
- Each can handle ~2 req/s → ~6 req/s total
- Expect ~1/3 the latency of Scenario 1

Scenario 3: Breaking the System

Same architecture as Scenario 2, but:

- VUs ramp from 100 → 300 → 500 → 800
- Containers limited to 64MB RAM, 0.5 CPU each
- k6 thresholds define pass/fail:
  - p(95) < 2 seconds
  - error rate < 10%

Watch for the inflection point where the system stops coping.

Activity Time

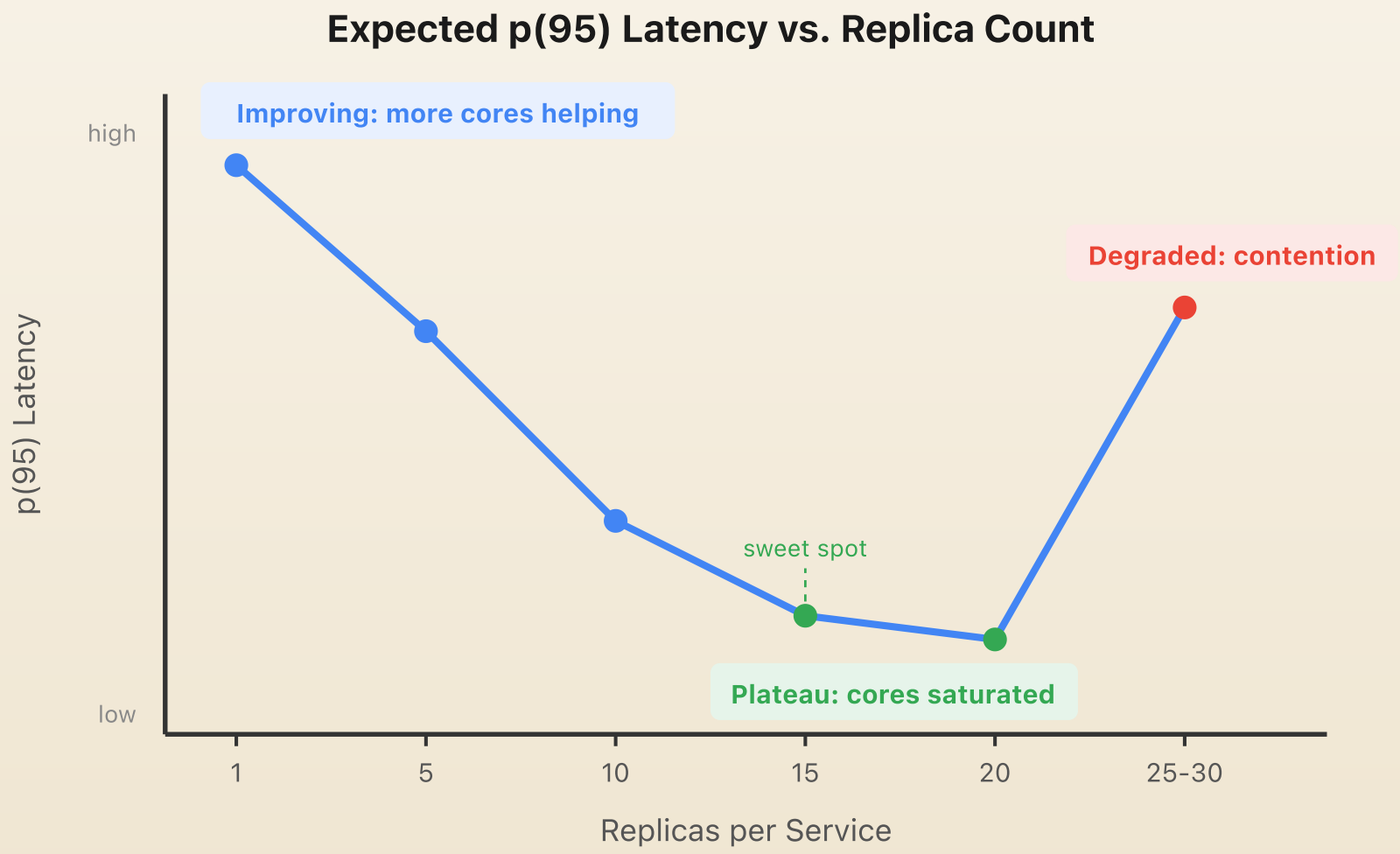
Scaling Experiment Debrief

You ran the load test at r=5, 10, 15, 20, 25, 30 replicas per service.

Questions:

- Where did p(95) stop improving?
- Where did it get **worse**?
- Why?

The Scaling Curve



Past the sweet spot, replicas compete for CPU cores and performance degrades.

Why More Replicas Made Things Worse

- Each replica does a CPU-bound busy-wait for 500ms
- That requires a real CPU core for the full duration
- Past your machine's core count, replicas compete for CPU time
- The OS time-slices, making each replica's 500ms take longer
- More replicas = more context switches = worse for everyone

This is the difference between concurrency and parallelism.

Scaling out past your physical resource limit makes things worse, not better.

So What Do You Do?

When adding replicas on one machine stops helping:

- **Vertical scaling**: bigger machine, more cores
- **Horizontal scaling**: more machines, each with their own cores
- **Optimize the work**: can you reduce the 500ms? Cache results? Make it async?

In production, this is why:

- Kubernetes clusters span multiple nodes
- Cloud platforms charge by the core
- Autoscalers need both replica count **and** node count limits

Key Takeaways

1. A load test that doesn't stress the server can't reveal architecture differences
2. CPU-bound work creates real contention; async I/O does not
3. Load balancing helps – but only up to the physical resource limit
4. Past that limit, adding replicas makes things worse
5. Use SLOs + load testing to find your capacity ceiling

Closing Question

> Your system handles 6 req/s across 3 replicas on one machine. You need to handle 60 req/s. What is your plan?

Full solution with step-by-step math – look only after you have done your own analysis.